INSTITUTO FEDERAL DO MARANHÃO
Ciência da computação

Introdução à Programação Orientada a Objetos

Prof. Dr. Welington Galvão Rodrigues
welington.rodrigues@ifma.edu.br

2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão

Apresentação



- Graduado em Sistemas de Informação - FACIMP.
- Mestrado e Doutorado em Ciências da Computação - UFG (Sistemas inteligentes e aplicações).
- Autor de Softwares com registro no INPI.
- Desenvolvedor com experiência em desenvolvimento de sistemas de informação.

Endereço do Curriculum Lattes:
http://lattes.cnpq.br/7647778556346999

Agenda

- Paradigmas de programação
- Conceitos da POO
- Ecossistema Java (JDK/JRE/JVM)
- Instalação do ambiente
- Primeiro programa Java
- Conclusão

Apresentação da Disciplina

Agenda

- Paradigmas de programação
- Conceitos da POO
- Ecossistema Java (JDK/JRE/JVM)
- Instalação do ambiente
- Primeiro programa Java
- Conclusão

Introdução

Introdução

Paradigma procedural

Ex: Cálculo da média da nota final.

```python
# Cálculo de média final e verificação de aprovação

# Solicita as notas do usuário
print("=== CÁLCULO DE MÉDIA FINAL ===")
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))
nota3 = float(input("Digite a terceira nota: "))
nota4 = float(input("Digite a quarta nota: "))

# Calcula a média
media = (nota1 + nota2 + nota3 + nota4) / 4

# Exibe a média
print(f"\nMédia final: {media:.1f}")

# Verifica se foi aprovado ou reprovado
if media >= 6:
    print("Situação: APROVADO")
else:
    print("Situação: REPROVADO")
```


Thor 100/100 HP
25/50 HP
-15 DAMAGE!
> Thor atacou Goblin!

Introdução

Paradigma procedural

```
# DADOS (dados soltos e desconexos)
# Imagine se fossem 100 personagens! Teríamos que gerenciar vários índices.
nomes = [None] * 2
vidas = [0.0] * 2
ataques = [0] * 2

# Cadastro do Guerreiro (Índice 0)
nomes[0] = "Thor"
vidas[0] = 100.0
ataques[0] = 15

# Cadastro do Monstro (Índice 1)
nomes[1] = "Goblin"
vidas[1] = 50.0
ataques[1] = 5
```

Introdução

Paradigma procedural

```python
# PROCEDIMENTO / FUNÇÃO
# A função precisa receber todos os parametros
def realizar_ataque(atacante, vitima, n, v, a):
    print(f"\n> {n[atacante]} atacou {n[vitima]}!")
    v[vitima] = v[vitima] - a[atacante]

    if v[vitima] < 0:
        v[vitima] = 0


def mostrar_status(indice, n, v):
    print(f"Personagem: {n[indice]} | Vida: {v[indice]}")
```

Introdução

Paradigma procedural

```
# PROGRAMA PRINCIPAL
print("--- Início do Jogo ---")
mostrar_status(0, nomes, vidas)
mostrar_status(1, nomes, vidas)

# AÇÃO (Processamento)
# Note como precisamos passar VÁRIOS parâmetros
realizar_ataque(0, 1, nomes, vidas, ataques)

print("\n--- Após o Ataque ---")
mostrar_status(1, nomes, vidas)
```

Conceituação teórica

Conceituação teórica

Programação orientada a objetos - POO

O que é paradigma?

Conceituação teórica

Programação orientada a objetos - POO

O que é paradigma? De acordo com o dicionário Michaelis, paradigma significa “algo que serve de exemplo ou modelo; padrão”

Programação orientada a objetos - POO

Programação Orientada a Objetos (POO) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados na forma de campos (atributos) e códigos na forma de procedimentos (métodos).

Dah & Nygaard

Programação orientada a objetos - POO

Programação Orientada a Objetos (POO) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados na forma de campos (atributos) e códigos na forma de procedimentos (métodos).

Dah & Nygaard

Conceituação teórica

Vantagens em relação ao procedural

- Reusabilidade de Código
- Manutenibilidade (Fácil Manutenção)
- Proximidade com o Mundo Real

Conceituação teórica

Quadro comparativo

Aspecto	Procedural	Orientado a Objetos
Organização	Funções e procedimentos	Classes e objetos
Dados	Variáveis globais/parâmetros	Atributos encapsulados nos objetos
Foco	*O que fazer* (ações)	*Quem faz* (entidades)
Reúso	Funções reutilizáveis	Herança e composição
Manutenção	Pode se tornar difícil (código espaguete)	Mais modular e fácil de manter
Exemplo	Calcular salário com funções	Objeto `Funcionario` com método `calcularSalario()`

Conceituação teórica

Princípios da POO - A.E.H.P

- Abstração
- Encapsulamento
- Herança
- Polimorfismo



Classes e objetos

Conceituação teórica

Classes e objetos

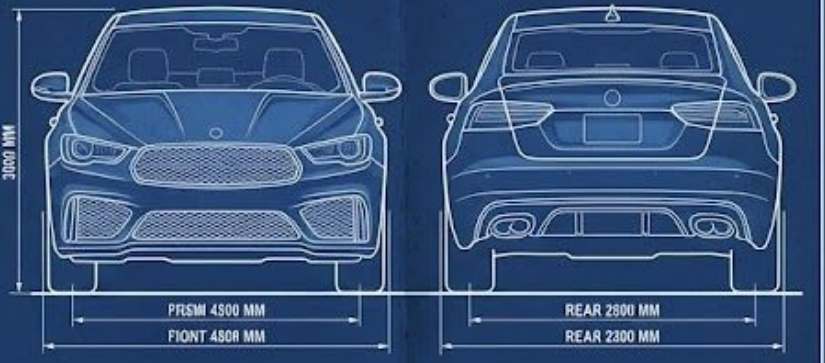


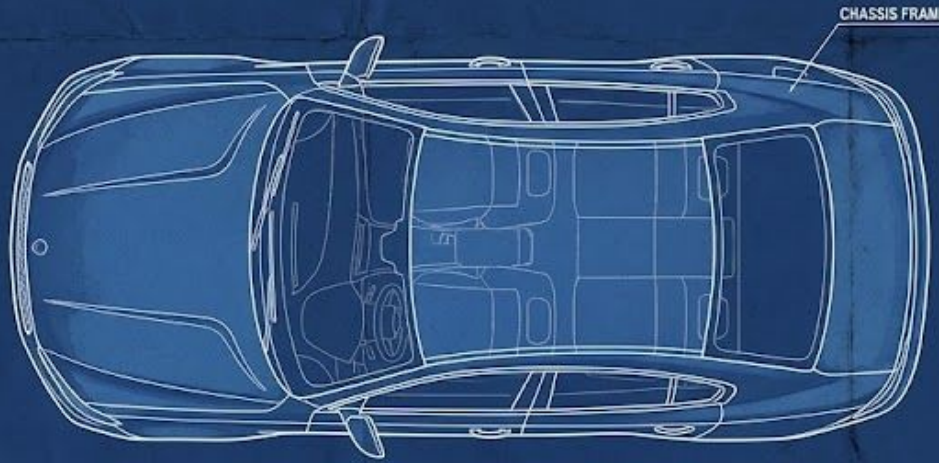
Classes e objetos

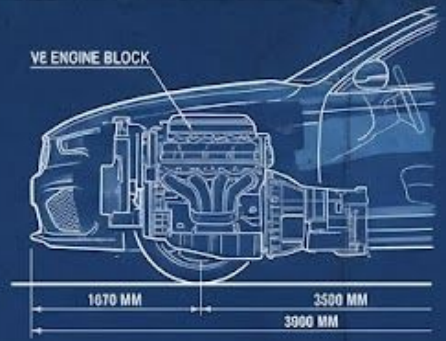
Classes

A classe atua como um modelo ou um plano para criar objetos. É uma estrutura que define atributos (dados) e métodos (ações) que os objetos criados a partir dela possuirão.











PROJECT:
ORION SEDAN
DRAWING NO.
A-101
REV.
C

```java
public class Carro {

    // --- ATRIBUTOS (As Especificações do Blueprint) ---
    String modelo;
    String cor;
    int ano;
    boolean motorLigado;
    double velocidadeAtual;

    // --- MÉTODOS (O que o carro foi projetado para fazer) ---
    // Representam o COMPORTAMENTO do objeto

    void ligar() {
        if (motorLigado == false) {
            motorLigado = true;
            System.out.println("O " + modelo + " ligou o motor (Vrum!).");
        } else {
            System.out.println("O motor já está ligado.");
        }
    }

    void acelerar(double intensidade) {
        if (motorLigado) {
            velocidadeAtual = velocidadeAtual + intensidade;
            System.out.println("Acelerando... Velocidade atual: " + velocidadeAtual + " km/h");
        } else {
            System.out.println("Não é possível acelerar com o motor desligado!");
        }
    }

    void frear() {
        velocidadeAtual = 0;
        System.out.println("O carro parou.");
    }
}
```

INSTITUTO FEDERAL
Maranhão

Classes e objetos

Objetos

Um objeto é uma instância concreta de uma classe. Ele é uma entidade que possui seus próprios valores para os atributos definidos na classe e pode executar os métodos especificados na classe. Cada objeto é independente dos outros, o que significa que eles podem armazenar diferentes valores de atributos e realizar ações de forma independente.


PROJECT: ORION SEDAN - CLASS BLUEPRINT
ORION SEDAN
A-101
INSTANCE 1: RED
(Modelo: Orion, Cor: Vermelho)
INSTANCE 2: BLUE
(Modelo: Orion, Cor: Azul)
INSTANCE 3: SILVER
(Modelo: Orion, Cor: Prata)

```java
public class FabricaDeCarros {

    public static void main(String[] args) {

        System.out.println("--- Fabricando o Carro 1 ---");
        // A palavra-chave 'new' aloca espaço na memória para este objeto específico
        Carro carro1 = new Carro();

        // Definindo o ESTADO inicial (Atributos)
        carro1.modelo = "Fusca";
        carro1.cor = "Azul";
        carro1.ano = 1980;

        // Executando COMPORTAMENTOS (Métodos)
        carro1.ligar();           // O objeto muda seu estado interno (motorLigado = true)
        carro1.acelerar(30.0);    // Velocidade vai para 30
        carro1.acelerar(20.0);    // Velocidade soma +20 = 50

        System.out.println("\n--- Fabricando o Carro 2 ---");

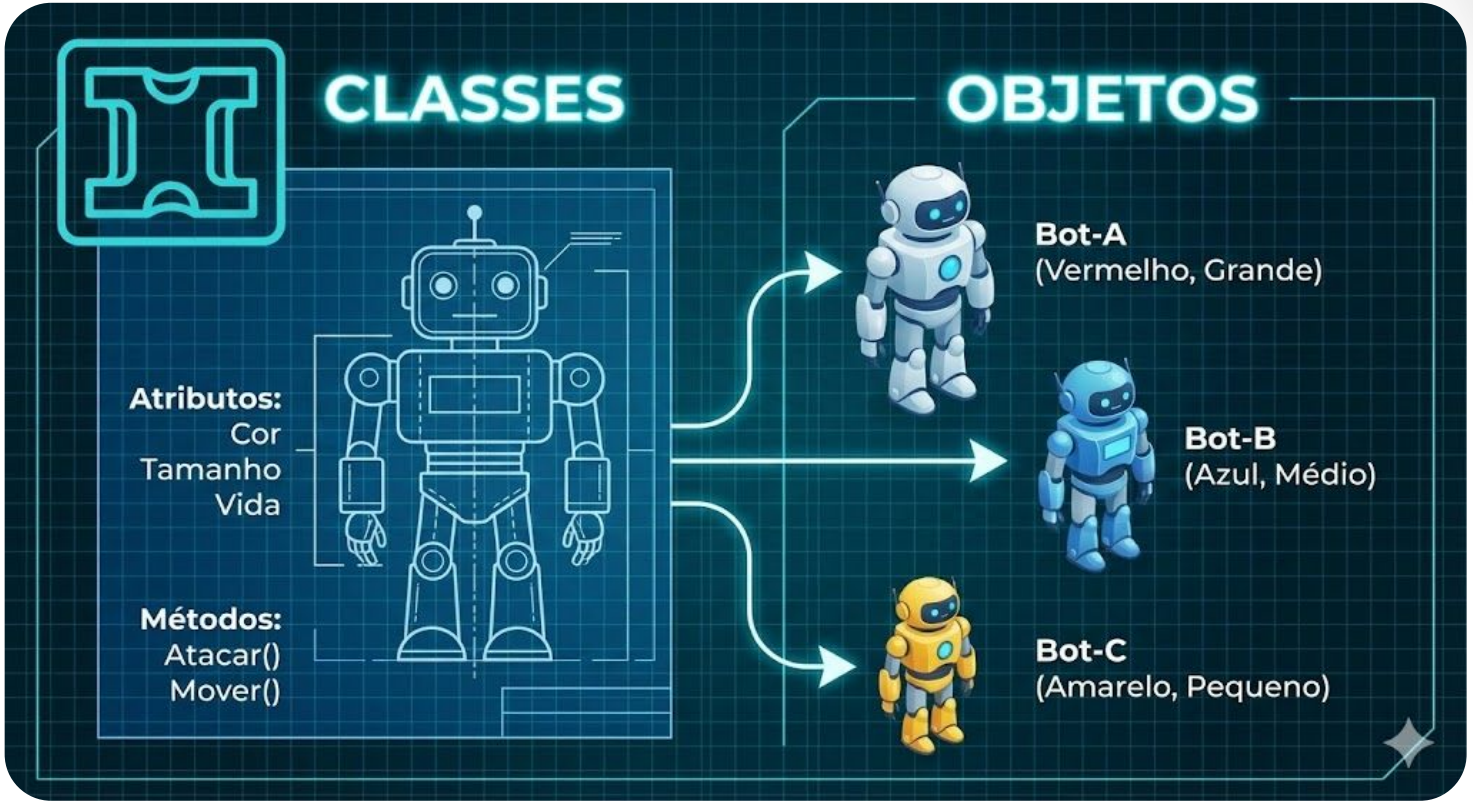
        Carro carro2 = new Carro(); // Novo espaço de memória, totalmente separado do carro1

        carro2.modelo = "Ferrari";
        carro2.cor = "Vermelha";

        // TENTATIVA DE ERRO: Acelerando sem ligar (testando a lógica da classe)
        carro2.acelerar(100.0); // O console deve mostrar: "Não é possível acelerar..."

        // Uso correto
        carro2.ligar();
        carro2.acelerar(200);
    }
}
```

Classes e objetos





Ecossistema Java

JVM · JRE · JDK · Bytecode · GC

Ecossistema Java

JVM — Java Virtual Machine

- Motor que carrega, verifica e executa o bytecode
- Gerencia memória, threads e segurança automaticamente
- Existe para Windows, Linux e macOS — garante portabilidade

Ecossistema Java

JRE — Java Runtime Environment

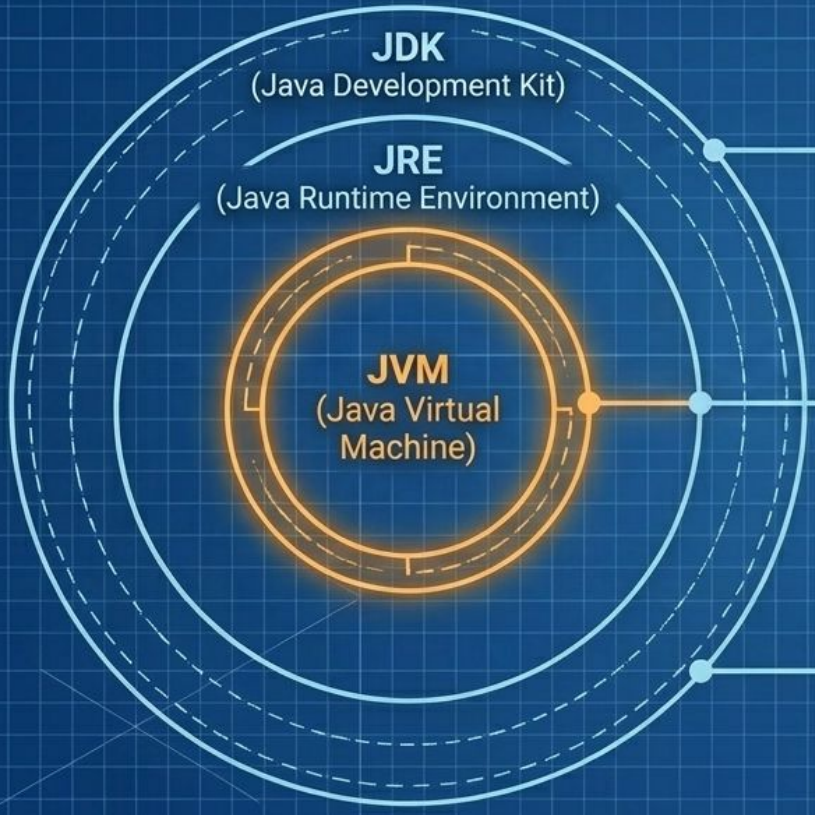
- Ambiente necessário para EXECUTAR programas Java
- Contém: JVM + bibliotecas padrão (java.lang, java.util...)
- Analogia: leitor de PDF — abre documentos, mas não os cria

Ecossistema Java

JDK — Java Development Kit

- Kit completo para DESENVOLVER em Java
- Contém: JRE + compilador javac + depurador jdb + ferramentas
- Analogia: Word — permite abrir E criar documentos

O Ecossistema Java: Desmistificando as Siglas



JVM: O coração lógico. O computador virtual que executa o `Bytecode` na máquina física.

JRE: O ambiente de execução para o usuário final. Contém a JVM + as Bibliotecas Base do Java. (Se você só quer rodar um programa, precisa apenas disso).

JDK: O kit do construtor. Contém o JRE + Ferramentas de Desenvolvimento (`javac`, bibliotecas avançadas). (Se você quer criar um programa, precisa disso).

NotebookLM

Ecossistema Java

Bytecode e WORA

- .java → javac → .class (bytecode) → JVM → execução
- Bytecode é independente de plataforma — roda em qualquer JVM
- "Write Once, Run Anywhere" (Sun Microsystems, 1995)

O Motor Híbrido: Compilação e Interpretação

Fase de Edição



`Livro.java`

Código-Fonte
(Escrito por você)

Fase de Compilação



`javac`
(Compilador)

Traduz a sintaxe e verifica erros

A Ponte



`Livro.class`

Bytecode. O segredo do Java. Um código intermediário, independente de plataforma.

Fase de Execução



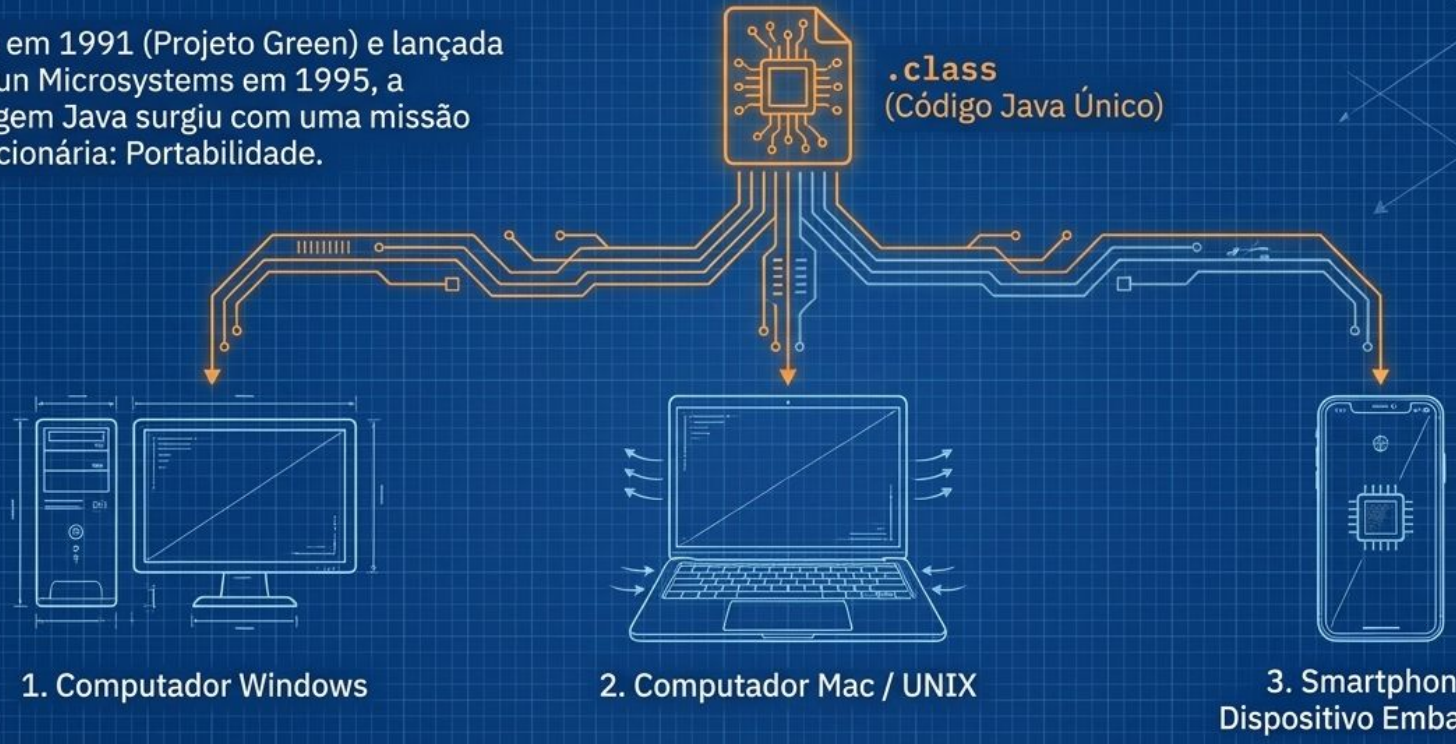
`JVM`
(Máquina Virtual Java)

Interpretador. Lê o bytecode e o traduz para a máquina física real no momento da execução.

NotebookLM

A Promessa do Java: Write Once, Run Anywhere

Criada em 1991 (Projeto Green) e lançada pela Sun Microsystems em 1995, a linguagem Java surgiu com uma missão revolucionária: Portabilidade.



Antes do Java, um programa precisava ser reescrito e recompilado para cada tipo de máquina. O Java eliminou essa barreira, permitindo que o mesmo código rodasse em qualquer hardware, impulsionando a expansão da Internet.

NotebookLM

Ecossistema Java

Garbage Collector (GC)

- Gerenciamento AUTOMÁTICO de memória pela JVM
- Objetos sem referência são coletados e a memória liberada
- Vantagem: sem memory leaks manuais — foco na lógica do negócio

Instalação do Ambiente

JDK 17+ e IDE (Eclipse / VS Code)

Instalação do Ambiente

Passo 1 — Instalar o JDK 17+

- Acesse: https://adoptium.net → Temurin 17 (LTS)
- Instalar e marcar "Add to PATH" durante a instalação
- Verificar: java -version e javac -version no terminal

Instalação do Ambiente

Passo 2 — Instalar a IDE

- Eclipse IDE → eclipse.org/downloads (recomendado)
- VS Code → code.visualstudio.com + Extension Pack for Java
- Criar projeto: File → New → Java Project → JRE 17



Primeiro Programa Java

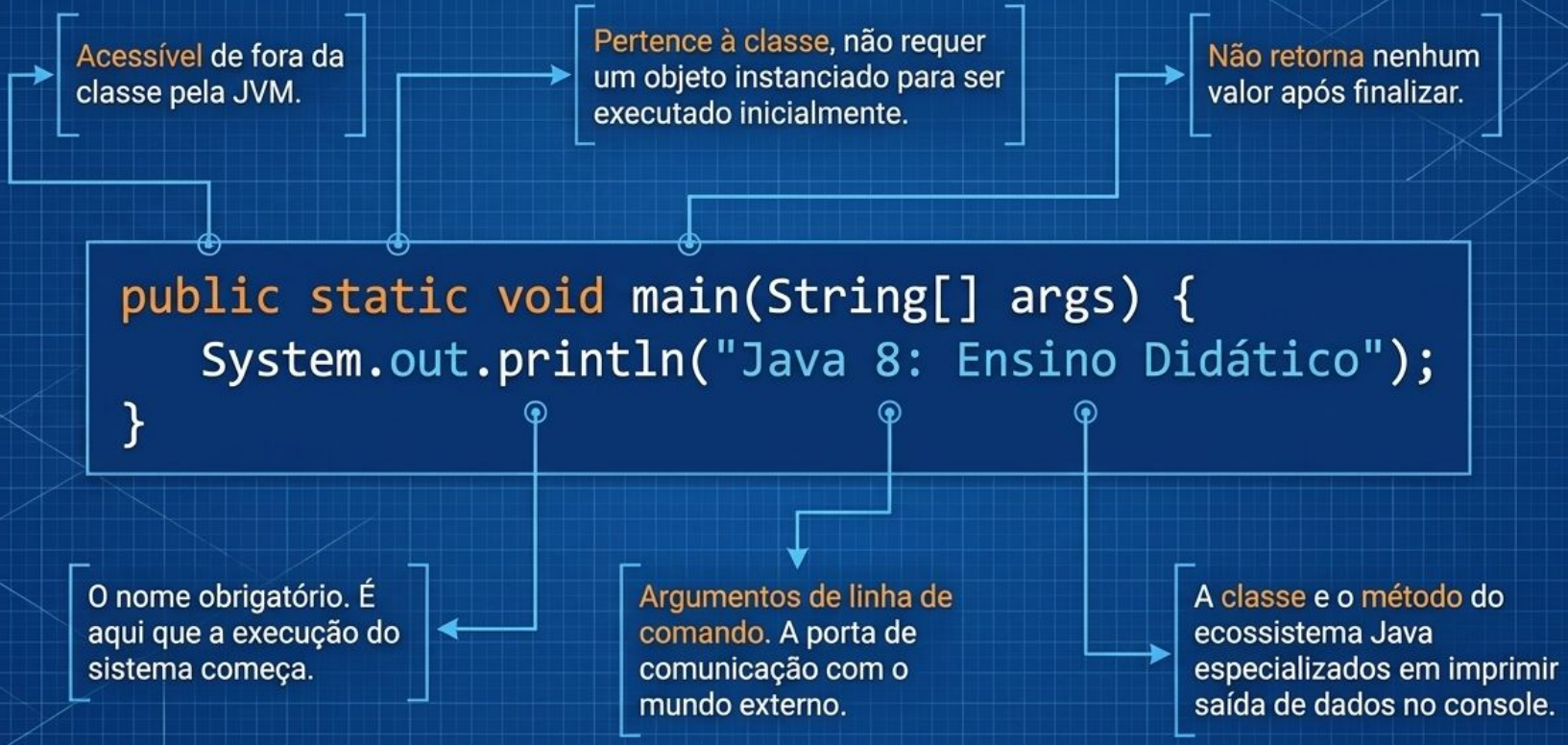
Estrutura, compilação e execução

Primeiro Programa Java

Estrutura de um programa Java

- package — organiza classes em namespaces (como pastas)
- public class NomeClasse — nome deve ser igual ao arquivo .java
- public static void main(String[] args) — ponto de entrada da JVM

A Porta de Entrada: Dissecando o Método **main**



NotebookLM

Primeiro Programa Java

Live coding — Definindo a classe Aluno

```java
// Arquivo: Aluno.java
public class Aluno {
    String nome;   // atributo
    int idade;  double nota;

    void exibirDados() {  // metodo
        System.out.println(nome + " | " + nota);
    }
}
```

Primeiro Programa Java

Live coding — Criando objetos na classe Main

```
public static void main(String[] args) {
    Aluno a1 = new Aluno();
    a1.nome = "Maria";    a1.nota = 8.5;
    a1.exibirDados();  // chama o metodo do objeto
}
```

Conclusões

Conclusões

- Paradigmas de programação
- Conceitos da POO (Classe, Objeto, Atributo, Método)
  - Os 4 Pilares: A.E.H.P
- Ecossistema Java: JVM, JRE, JDK
- Bytecode, WORA e Garbage Collector
- Primeiro programa Java

INSTITUTO FEDERAL DO MARANHÃO
Ciência da computação

Introdução à Programação Orientada a Objetos

Prof. Dr. Welington Galvão Rodrigues
welington.rodrigues@ifma.edu.br

2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão